

Quiz Assignment

Complete Interview Preparation Guide

Laravel · PHP · Eloquent ORM · Strategy Pattern

A from-zero walkthrough of every concept, every file, and every likely interview question — written for someone new to Laravel.

Table of Contents

1. **Project Overview** — what the app does
2. **The Big Picture** — how Laravel apps work
3. **What is Laravel?** — beginner orientation
4. **What is an ORM?** — Eloquent explained
5. **The Request Lifecycle** — browser → response, step by step
6. **Routing** — `routes/web.php` deeply explained
7. **Controllers** — every controller and method
8. **Models & Database** — schema, relationships, migrations
9. **Eloquent in Practice** — the patterns used in this app
10. **The Question Type System** — strategy pattern + registry
11. **The Service Container & Dependency Injection**
12. **File Uploads & Storage**
13. **Validation**
14. **Views & Blade**
15. **The Complete Request Walkthrough** — start to finish
16. **Glossary of Terms**
17. **50+ Interview Questions with Answers**
18. **Final Cheat Sheet & Talking Points**

1. Project Overview

This project is a **dynamic quiz system** built with Laravel. Admins can create quizzes, add questions of **five different types**, and students can take those quizzes and get auto-scored.

What the app can do

- **Quiz CRUD** — create, edit, delete quizzes (title + description).
- **Add questions** with rich body text, optional image, optional video URL, and a **marks** value.
- **Five question types**: Binary (Yes/No), Single Choice, Multiple Choice, Number Input (with tolerance), Text Input (with match modes).
- **Take a quiz** — student sees questions, submits answers, gets an auto-graded result.
- **Review results** — per-question marks awarded.

The two big ideas you must explain

INTERVIEW HEADLINE #1

Strategy Pattern via a Type Registry. Different question types behave differently. Instead of giant **if/else** chains scattered across the controller, each type is a class implementing a common **QuestionType** interface. A registry maps a slug like **"binary"** to the right handler. Adding a new type = one new class + one line in the service provider.

INTERVIEW HEADLINE #2

Single-table polymorphism with a JSON **config column.** Rather than 5 different question tables, all questions live in one **questions** table. Type-specific settings (e.g. tolerance for a number, match mode for text) live in a flexible JSON **config** column. Simple schema, infinite extensibility.

The technology stack

Layer	Technology	Role
Backend framework	Laravel 13	Routes, controllers, ORM, views
Language	PHP 8.x	Server-side logic
ORM	Eloquent	Maps PHP objects ↔ database rows
Database	SQLite	Single-file local DB (<code>database/database.sqlite</code>)
Frontend templating	Blade	HTML with PHP-friendly syntax
Asset bundler	Vite + Tailwind	CSS/JS compilation, dev server
File storage	Local disk (public)	Uploaded images for questions/options

2. The Big Picture: How a Laravel App Works

Before diving into code, let's understand what happens between "user clicks a link" and "user sees a page". **Every Laravel app follows the same lifecycle.**

The five-step lifecycle

- 1 **Browser sends an HTTP request** to your server (e.g. `GET /quizzes`).
- 2 **Laravel's router** looks at the URL + verb and decides which controller method to call.
- 3 **The controller method runs** — it talks to the database via the ORM, applies business logic, and prepares data.
- 4 **The controller returns a response** — usually by rendering a Blade view (HTML) or redirecting.
- 5 **The browser receives the HTML** and renders it.

MENTAL MODEL

Think of Laravel as a **receptionist**. The browser walks in saying "I want page X". The receptionist (router) checks a list, finds the right specialist (controller), the specialist fetches what's needed from the filing cabinet (database), assembles a document (view), and hands it back to the visitor.

The MVC pattern

Laravel is built around **MVC** — Model, View, Controller:

Letter	Stands for	In this project	Plain English
M	Model	<code>app/Models/Quiz.php</code> , <code>Question.php</code> , etc.	The data and rules about it (a Quiz has a title, a Quiz has many Questions).
V	View	<code>resources/views/quizzes/*.blade.php</code>	The HTML the user sees.
C	Controller	<code>app/Http/Controllers/QuizController.php</code>	The traffic cop. Receives the request, asks the model for data, sends data to the view.

3. What is Laravel?

Laravel is a PHP framework. A framework is a collection of pre-written code that solves common problems so you don't have to. Routing, database access, templating, validation, file uploads, sessions, security — all included.

Why Laravel exists

Without a framework, building a web app means you write everything yourself: parsing HTTP requests, escaping HTML, querying the database safely, handling cookies, validating forms. Tedious and error-prone. Laravel gives you tested, secure defaults so you focus on your application's logic.

The Laravel folders you actually touch

Folder	What lives there
<code>routes/web.php</code>	URL → controller mappings
<code>app/Http/Controllers/</code>	Controllers — handle requests
<code>app/Models/</code>	Eloquent models — represent DB tables
<code>app/Providers/</code>	Bootstrap code (e.g. wiring up the registry)
<code>database/migrations/</code>	Schema definitions (creates tables)
<code>resources/views/</code>	Blade templates (HTML)
<code>config/</code>	Configuration files
<code>storage/</code>	Logs, uploaded files, compiled views
<code>public/</code>	Web root — only files in here are directly accessible via URL

What is Artisan?

`php artisan` is Laravel's command-line tool. You'll see it used like:

```
php artisan serve           # start dev server on :8000
php artisan migrate         # apply migrations to DB
php artisan make:controller Foo # generate a controller file
php artisan route:list      # show all registered routes
php artisan tinker          # interactive REPL on your app
```

4. What is an ORM? (Eloquent Explained)

ORM = Object-Relational Mapping. It's a layer that lets you treat database rows as PHP objects, instead of writing raw SQL.

Without an ORM (raw SQL)

```
$pdo = new PDO("sqlite:database.sqlite");
$stmt = $pdo->prepare("SELECT * FROM quizzes WHERE id = ?");
$stmt->execute([5]);
$row = $stmt->fetch(PDO::FETCH_ASSOC);
echo $row['title'];
```

With Eloquent (the ORM Laravel uses)

```
$quiz = Quiz::find(5);
echo $quiz->title;
```

Same result, much cleaner. Eloquent runs the SQL behind the scenes.

What Eloquent gives you

- **Models** — one PHP class per table. `Quiz` ↔ `quizzes` table.
- **Relationships** — express "a Quiz has many Questions" in code; Eloquent figures out the foreign keys.
- **Query builder** — chainable, expressive: `Quiz::where('title', 'Math')->latest()->get()`.
- **Mass assignment protection** — only fields in `$fillable` can be bulk-set.
- **Casts** — auto-convert DB values (e.g. JSON column → PHP array; `0/1` → `true/false`).
- **Eager loading** — solve the "N+1 query" problem with one line.

The convention magic

Eloquent uses **convention over configuration**. By default:

- Class `Quiz` → table `quizzes` (auto-pluralize).
- Primary key is `id`.
- Foreign key for "belongs to Quiz" is `quiz_id`.
- Timestamps `created_at` and `updated_at` auto-managed.

You can override any of these, but if you follow the convention, you write almost no configuration.

5. The Request Lifecycle in Detail

Let's trace what happens when a user opens `http://localhost:8000/quizzes` in their browser.

- 1 **Browser fires `GET /quizzes`** . The browser opens a TCP connection to your server.
- 2 **Web server (PHP built-in via `php artisan serve`) hands the request to `public/index.php`** . This is the only PHP file the web server ever runs — it's the front controller.
- 3 **Laravel boots**. The framework loads, reads `.env` , registers all **service providers** (including `QuestionTypeServiceProvider` which sets up our registry).
- 4 **The Router runs middleware and matches the route**. Middleware are layers like authentication, CSRF protection, session handling. After they pass, it finds `Route::resource('quizzes', QuizController::class)` which maps `GET /quizzes` → `QuizController@index` .
- 5 **Laravel constructs the controller**. Looking at the constructor, if it needs anything (like `QuestionTypeRegistry`), the service container builds and injects it.
- 6 **`index()` executes**. It runs `Quiz::withCount('questions')->latest()->get()` which Eloquent translates into SQL and runs against SQLite.
- 7 **The view renders**. `view('quizzes.index', compact('quizzes'))` compiles `resources/views/quizzes/index.blade.php` with the `$quizzes` variable.
- 8 **HTML is returned**. Laravel sends the rendered HTML back through any "after" middleware, then to the browser.
- 9 **Browser renders**. Done.

LIKELY QUESTION

"What happens between the URL and the page?" — Walk them through these 9 steps. Even high-level acknowledgement of front-controller, service container boot, middleware chain, route match, controller dispatch, view render is impressive.

6. Routing Explained

Routes live in `routes/web.php`. They tell Laravel: "When the browser asks for THIS URL with THIS method, run THAT controller method."

The actual file

```
Route::redirect('/', '/quizzes');

Route::resource('quizzes', QuizController::class);

Route::post('quizzes/{quiz}/questions', [QuestionController::class, 'store'])
    ->name('questions.store');
Route::get('questions/{question}/edit', [QuestionController::class, 'edit'])
    ->name('questions.edit');
Route::put('questions/{question}', [QuestionController::class, 'update'])
    ->name('questions.update');
Route::delete('questions/{question}', [QuestionController::class, 'destroy'])
    ->name('questions.destroy');

Route::get('quizzes/{quiz}/take', [AttemptController::class, 'start'])
    ->name('attempts.start');
Route::post('attempts/{attempt}/submit', [AttemptController::class, 'submit'])
    ->name('attempts.submit');
Route::get('attempts/{attempt}', [AttemptController::class, 'show'])
    ->name('attempts.show');
```

Line by line

```
Route::redirect('/', '/quizzes');
```

If anyone visits the root URL `/`, send them to `/quizzes`. Saves writing a controller for a one-liner redirect.

```
Route::resource('quizzes', QuizController::class);
```

This **one line** creates 7 routes — the standard RESTful set:

Verb	URI	Method	Name	Purpose
GET	/quizzes	index	quizzes.index	List all quizzes
GET	/quizzes/create	create	quizzes.create	Show "new quiz" form
POST	/quizzes	store	quizzes.store	Save the new quiz
GET	/quizzes/{quiz}	show	quizzes.show	Show one quiz
GET	/quizzes/{quiz}/edit	edit	quizzes.edit	Show edit form
PUT/PATCH	/quizzes/{quiz}	update	quizzes.update	Save edits
DELETE	/quizzes/{quiz}	destroy	quizzes.destroy	Delete the quiz

QuizController::class

This is **not** creating an instance. `::class` is a PHP language construct that returns the class name as a string: `"App\Http\Controllers\QuizController"`. We use it instead of writing the string ourselves because the IDE can autocomplete and refactor it safely.

Named routes — `->name('questions.store')`

A label for the route. Anywhere in the app you can write `route('questions.store', $quiz)` instead of hardcoding the URL. If the URL ever changes, every reference still works.

Route parameters — `{quiz}`, `{question}`, `{attempt}`

Anything in curly braces is a placeholder. Laravel passes the value to the controller. With **route-model binding** (see below), Laravel goes a step further: if the controller method declares `Quiz $quiz`, Laravel looks up the model by ID and passes the actual `Quiz` object — or returns 404 if not found.

ROUTE-MODEL BINDING (MUST KNOW!)

When a controller method signature says `public function show(Quiz $quiz)`, Laravel sees the URL `/quizzes/5`, runs `Quiz::findOrFail(5)`, and hands the resulting model object to your method. **You never write the lookup yourself.** If `5` doesn't exist, the user gets a 404 automatically.

Why some routes are written manually instead of `Route::resource`

The "questions" routes are **nested under quizzes** (`/quizzes/{quiz}/questions`) and only some of the 7 RESTful actions are needed. Easier to write 4 lines than configure a partial nested resource.

7. Controllers in Detail

A controller is a class with public methods. Each method handles one type of HTTP request. The method's job is: **read the request, do work (usually DB), return a response**.

7.1 QuizController

`index()` — list quizzes

```
public function index()
{
    $quizzes = Quiz::withCount('questions')->latest()->get();
    return view('quizzes.index', compact('quizzes'));
}
```

- `Quiz::withCount('questions')` — fetch all quizzes AND for each one, count its questions (in a single SQL query, not N+1). Each quiz then has a `questions_count` integer property.
- `->latest()` — order by `created_at DESC`, newest first.
- `->get()` — execute the query, return a Collection of Quiz objects.
- `view('quizzes.index', compact('quizzes'))` — render `resources/views/quizzes/index.blade.php` and pass `$quizzes` into it. `compact('quizzes')` is shorthand for `['quizzes' => $quizzes]`.

`create()` — show "new quiz" form

```
public function create()
{
    return view('quizzes.create');
}
```

Renders the empty form. No DB work needed.

store() — save a new quiz

```
public function store(Request $request)
{
    $data = $request->validate([
        'title' => ['required', 'string', 'max:255'],
        'description' => ['nullable', 'string'],
    ]);

    $quiz = Quiz::create($data);

    return redirect()->route('quizzes.edit', $quiz)
        ->with('status', 'Quiz created. Add your questions below.');
```

- **Validate first.** If validation fails, Laravel auto-redirects back with errors. If it passes, `$data` contains only the validated fields.
- `Quiz::create($data)` — create a new row from the array (only fields in `$fillable` on the model are accepted, for security).
- **Redirect to the edit page**, not back to a "show" page. Why? Because creating a quiz is just step 1 — you still need to add questions. Flash a one-time success message via `->with('status', ...)`.

edit() — show the editor

```
public function edit(Quiz $quiz, QuestionTypeRegistry $registry)
{
    $quiz->load('questions.options');

    return view('quizzes.edit', [
        'quiz' => $quiz,
        'types' => $registry->options(),
        'registry' => $registry,
    ]);
}
```

- **Two things injected:** `Quiz $quiz` (route-model binding from the URL) and `QuestionTypeRegistry $registry` (from the service container).
- `$quiz->load('questions.options')` — eager-load questions AND each question's options. Without this, the view would trigger one extra query per question (the N+1 problem).
- **Three variables to the view:** the quiz itself, a slug→label map for the question-type dropdown, and the full registry object so the view can ask for type-specific partials.

update() — save quiz edits

```
public function update(Request $request, Quiz $quiz)
{
    $data = $request->validate([
        'title' => ['required', 'string', 'max:255'],
        'description' => ['nullable', 'string'],
    ]);

    $quiz->update($data);

    return redirect()->route('quizzes.edit', $quiz)->with('status', 'Quiz updated.');
```

Same pattern as `store`, but updating an existing model. `$quiz->update($data)` sets the new values and saves.

destroy() — delete a quiz (and clean up files)

```
public function destroy(Quiz $quiz)
{
    foreach ($quiz->questions as $question) {
        if ($question->image_path) {
            Storage::disk('public')->delete($question->image_path);
        }
        foreach ($question->options as $option) {
            if ($option->image_path) {
                Storage::disk('public')->delete($option->image_path);
            }
        }
    }

    $quiz->delete();

    return redirect()->route('quizzes.index')->with('status', 'Quiz deleted.');
```

WHY THE MANUAL FILE CLEANUP?

The DB rows cascade-delete via foreign key constraints (`cascadeOnDelete` in the migration). But uploaded files on disk are **not** tracked by Eloquent — they would be orphaned forever. So we delete them manually before the row goes.

7.2 QuestionController

This controller manages individual questions. Critically, it uses the **QuestionTypeRegistry** to delegate type-specific work — it never has an `if/else` over question types.

Constructor

```
public function __construct(private QuestionTypeRegistry $registry) {}
```

Constructor property promotion (PHP 8). Laravel injects the registry on every request. The controller never knows about specific type classes.

store() — create a question

```
public function store(Request $request, Quiz $quiz)
{
    $base = $this->validateBase($request);
    $type = $this->registry->for($base['type']);

    $imagePath = $request->hasFile('image')
        ? $request->file('image')->store('questions', 'public')
        : null;

    $question = $quiz->questions()->create([
        'type' => $base['type'],
        'body_html' => $base['body_html'],
        'image_path' => $imagePath,
        'video_url' => $base['video_url'],
        'marks' => $base['marks'],
        'position' => $quiz->questions()->max('position') + 1,
        'config' => $type->buildConfig($request->all()),
    ]);

    $type->persistOptions($question, $request->all());
    $this->persistOptionImages($question, $request);

    return redirect()->route('quizzes.edit', $quiz)->with('status', 'Question added.');
```

Step by step:

1. **Validate common fields** via `validateBase()` (body, marks, video, image, type).
2. **Look up the type handler** from the registry — `$type` is now a `BinaryType`, `MultipleChoiceType`, etc.
3. **If an image was uploaded**, store it on the public disk and capture the relative path.
4. **Create the question** via the relationship — `$quiz->questions()->create([...])` auto-fills `quiz_id`.
5. **Position** = current max + 1 → new questions go at the end.
6. **config** = whatever the type handler returns from `buildConfig()`. Stored as JSON in the DB.
7. **Delegate option saving** — the type knows how (multiple choice creates 4 rows; binary creates 0).
8. **Handle option images** separately.
9. **POST/Redirect/GET** — redirect to the editor with a flash message.

edit() — show edit form

```
public function edit(Question $question)
{
    $question->load('options');
    return view('questions.edit', [
        'question' => $question,
        'type' => $this->registry->for($question->type),
    ]);
}
```

Pass the question and its type handler to the view. The view uses the handler's `editorView()` method to include the right Blade partial.

update()

Same pattern as `store` but with extra image logic for three cases:

- **New image uploaded** → delete old, store new.
- **"Remove image" checkbox checked** → delete file, null the column.
- **Neither** → leave the existing image alone.

destroy()

Captures `$quizId` first (we need it for the redirect after deletion), deletes all uploaded files, then deletes the question. Option rows cascade via FK constraint.

validateBase() — DRY validation helper

```
private function validateBase(Request $request, bool $isUpdate = false): array
{
    $rules = [
        'body_html' => ['required', 'string'],
        'video_url' => ['nullable', 'url'],
        'marks' => ['required', 'integer', 'min:1'],
        'image' => ['nullable', 'image', 'max:4096'],
    ];

    if (! $isUpdate) {
        $rules['type'] = ['required', 'string', 'in:'.implode(',', array_keys($this->registry->for('question')))];
    }

    return $request->validate($rules);
}
```

CLEVER BIT

The `type` validation rule is generated **from the registry**:

`in:binary,single_choice,multiple_choice,number,text` . Add a new type → validation auto-extends. Open/Closed Principle in action.

`persistOptionImages()`

Form sends option images as an array (`option_images[0]` , `option_images[1]` , ...). For each option, check if there's a file at the same index, replace if so.

7.3 AttemptController

Handles a student taking a quiz. Three actions: `start` , `submit` , `show` .

`start()` — begin an attempt

```
public function start(Quiz $quiz)
{
    abort_if($quiz->questions()->count() === 0, 404, 'This quiz has no questions yet.');
```

`$quiz->load('questions.options');`

```
    $attempt = $quiz->attempts()->create([
        'started_at' => now(),
        'max_score' => $quiz->totalMarks(),
    ]);

    return view('attempts.take', [
        'quiz' => $quiz,
        'attempt' => $attempt,
        'registry' => $this->registry,
    ]);
}
```

- **Guard:** 404 if the quiz has no questions.
- **Eager-load** questions and options.
- **Create an Attempt record** — captures start time and max possible score.
- Render the "take quiz" page.

submit() — score the answers

```
public function submit(Request $request, Attempt $attempt)
{
    abort_if($attempt->isSubmitted(), 403, 'This attempt has already been submitted.');
```

 \$attempt->load('quiz.questions.options');

 \$raw = (array) \$request->input('answers', []);

 \$total = 0.0;

 foreach (\$attempt->quiz->questions as \$question) {

 \$type = \$this->registry->for(\$question->type);

 \$payload = \$type->normalizePayload(\$raw[\$question->id] ?? null);

 \$marks = \$type->evaluate(\$question, \$payload);

 \$attempt->answers()->create([

 'question_id' => \$question->id,

 'payload' => \$payload,

 'marks_awarded' => \$marks,

]);

 \$total += \$marks;

 }

 \$attempt->update([

 'submitted_at' => now(),

 'score' => \$total,

 'max_score' => \$attempt->quiz->totalMarks(),

]);

 return redirect()->route('attempts.show', \$attempt);

}

The scoring loop is the heart of the app. For every question:

1. **Get the right type handler.**
2. **Normalize** the raw form input into a clean array.
3. **Evaluate** — the type returns marks awarded (0 to `$question->marks`).
4. **Save an Answer row** with the payload and marks.
5. **Accumulate** the total.

After the loop, update the Attempt with the final score and submission time.

NOTICE: ZERO IF/ELSE OVER QUESTION TYPES

The scoring loop is type-agnostic. It just calls methods on the type handler. This is the strategy pattern paying off — the controller would not change if you added 50 more question types tomorrow.

show() — display the result

```
public function show(Attempt $attempt)
{
    abort_unless($attempt->isSubmitted(), 404);
    $attempt->load('quiz', 'answers.question.options');

    return view('attempts.result', [
        'attempt' => $attempt,
        'registry' => $this->registry,
    ]);
}
```

Eager-load everything needed for the review page; render the result view.

8. Models & Database

The schema (5 tables)

Table	Columns	Relationships
quizzes	id, title, description, timestamps	has many questions, has many attempts
questions	id, quiz_id (FK), type, body_html, image_path, video_url, marks, config (JSON), position, timestamps	belongs to quiz, has many options
options	id, question_id (FK), label, image_path, is_correct, position, timestamps	belongs to question
attempts	id, quiz_id (FK), started_at, submitted_at, score, max_score, timestamps	belongs to quiz, has many answers
answers	id, attempt_id (FK), question_id (FK), payload (JSON), marks_awarded, timestamps	belongs to attempt, belongs to question

The Quiz model

```
class Quiz extends Model
{
    use HasFactory;

    protected $fillable = ['title', 'description'];

    public function questions(): HasMany
    {
        return $this->hasMany(Question::class)->orderBy('position');
    }

    public function attempts(): HasMany
    {
        return $this->hasMany(Attempt::class);
    }

    public function totalMarks(): int
    {
        return (int) $this->questions()->sum('marks');
    }
}
```

- **\$fillable** — only these fields can be mass-assigned via **create()** or **update()** . Security against attackers POSTing extra fields.

- `questions()` — defines the **has-many** relationship. The `orderBy('position')` means whenever you load questions, they come out in order.
- `totalMarks()` — a helper to sum marks across all questions. Used when starting/submitting an attempt.

The Question model

```
class Question extends Model
{
    protected $fillable = ['quiz_id', 'type', 'body_html', 'image_path',
                          'video_url', 'marks', 'config', 'position'];

    protected $casts = [
        'config' => 'array',
        'marks' => 'integer',
        'position' => 'integer',
    ];

    public function quiz(): BelongsTo { return $this->belongsTo(Quiz::class); }
    public function options(): HasMany { return $this->hasMany(Option::class)->orderBy('pos
}
}
```

THE CASTS ARRAY — IMPORTANT!

The `config` column stores JSON in the DB. With `'config' => 'array'` in `$casts`, Eloquent **automatically** decodes it into a PHP array when you read, and encodes it back when you save. You never call `json_encode` or `json_decode` yourself.

The migration that creates `questions`

```
Schema::create('questions', function (Blueprint $table) {
    $table->id();
    $table->foreignId('quiz_id')->constrained()->cascadeOnDelete();
    $table->string('type');
    $table->longText('body_html');
    $table->string('image_path')->nullable();
    $table->string('video_url')->nullable();
    $table->unsignedInteger('marks')->default(1);
    $table->json('config')->nullable();
    $table->unsignedInteger('position')->default(0);
    $table->timestamps();
    $table->index(['quiz_id', 'position']);
});
```

- `foreignId('quiz_id')->constrained()->cascadeOnDelete()` — creates a foreign key to `quizzes.id`. If the parent quiz is deleted, all its questions are auto-deleted by the database.
- `json('config')` — native JSON column type.

- `index(['quiz_id', 'position'])` — composite index because we frequently query questions for a quiz, ordered by position. Speeds up that query massively.

What is a migration?

A migration is a versioned, code-based way to evolve the database schema. Instead of running raw `CREATE TABLE` SQL by hand, you write a PHP file and run `php artisan migrate`. Every team member, every server, gets the same schema. You can also `migrate:rollback` to undo.

9. Eloquent in Practice

Relationships

Type	Example	Use
hasMany	<code>Quiz::questions()</code>	One Quiz → many Questions
belongsTo	<code>Question::quiz()</code>	One Question → one Quiz (the inverse)

The query patterns used in this app

```
Quiz::withCount('questions')->latest()->get()
```

Fetches all quizzes, attaches a `questions_count` integer to each (in one efficient subquery), orders by `created_at` desc.

```
$quiz->load('questions.options')
```

Eager-load: fetch all questions for this quiz, AND all options for each question. **Three queries total** instead of $N+1$. The dot notation means "nested relationship".

```
$quiz->questions()->create([...])
```

Create a new Question that belongs to this quiz. Eloquent auto-sets `quiz_id = $quiz->id`. You don't pass it.

```
$quiz->questions()->max('position') + 1
```

Run a SQL `MAX(position)` on this quiz's questions. Used to set the position of the next question to "last + 1".

```
$question->options()->delete()
```

Delete every option belonging to this question — used when re-saving options on update.

```
$question->options->firstWhere('id', $pickedId)
```

This is on the **collection**, not the query. Once options are loaded into memory, find the one with that id without hitting the DB again.

The N+1 problem (very common interview question)

N+1 DEFINED

You fetch N parents in 1 query. Then you loop and access a relationship on each — triggering N extra queries. Total: N+1 queries. For 100 quizzes with questions, that's 101 SQL hits per page load.

Fix: Eager-load with `with()` or `load()` . Or use `withCount()` when you only need the count.

load VS loadMissing VS fresh

Method	Behavior
<code>\$model->load('rel')</code>	Always loads the relationship (re-runs the query)
<code>\$model->loadMissing('rel')</code>	Loads only if not already loaded
<code>\$model->fresh('rel')</code>	Returns a NEW model freshly fetched from DB with that relationship

10. The Question Type System (Strategy + Registry)

This is the most important architectural piece. Be ready to walk through it.

The problem it solves

Without this pattern, every method that touches questions would have a giant `if/else` over the type:

```
// THE MESS THIS CODE AVOIDS
if ($question->type === 'binary') { /* score binary */ }
elseif ($question->type === 'multiple_choice') { /* score MC */ }
elseif ($question->type === 'number') { /* score number */ }
// ... in store(), update(), submit(), every view ...
```

Adding a new type would mean editing 6+ files. Forgetting one breaks something silently.

The solution

Each type is a class implementing a `QuestionType` interface. A registry holds them. The controller looks up by slug and delegates.

The interface (the contract)

```
interface QuestionType
{
    public function slug(): string;
    public function label(): string;
    public function buildConfig(array $input): array;
    public function persistOptions(Question $q, array $input): void;
    public function normalizePayload(mixed $raw): array;
    public function evaluate(Question $q, array $payload): float;
    public function editorView(): string;
    public function inputView(): string;
    public function reviewView(): string;
}
```

Method	Plain English
slug()	"What's my unique short name?"
label()	"What's my human-readable name (for UI)?"
buildConfig(\$input)	"What type-specific settings should we save in the JSON config?"
persistOptions(\$q, \$input)	"Save my answer options however I see fit."
normalizePayload(\$raw)	"Clean up the raw user answer into a structured array."
evaluate(\$q, \$payload)	"Score this answer. Return marks awarded."
editorView()	"Which Blade partial renders my admin form fields?"
inputView()	"Which Blade partial renders me for the student?"
reviewView()	"Which Blade partial renders me on the result page?"

The registry

```

class QuestionTypeRegistry
{
    private array $types = [];

    public function register(QuestionType $type): self
    {
        $this->types[$type->slug()] = $type;
        return $this;
    }

    public function for(string $slug): QuestionType
    {
        if (! isset($this->types[$slug])) {
            throw new RuntimeException("Unknown question type: {$slug}");
        }
        return $this->types[$slug];
    }

    public function all(): array { return $this->types; }

    public function options(): array
    {
        $out = [];
        foreach ($this->types as $slug => $type) {
            $out[$slug] = $type->label();
        }
        return $out;
    }
}

```

Tiny class. Four methods:

- `register` — add a type. Returns `$this` for chaining.
- `for($slug)` — look up; throw if missing (fail fast).
- `all()` — return the whole map (used to build the validation rule dynamically).
- `options()` — return slug→label for dropdowns.

How it gets populated — the service provider

```
class QuestionTypeServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        $this->app->singleton(QuestionTypeRegistry::class, function () {
            return (new QuestionTypeRegistry)
                ->register(new BinaryType)
                ->register(new SingleChoiceType)
                ->register(new MultipleChoiceType)
                ->register(new NumberType)
                ->register(new TextType);
        });
    }
}
```

WHAT THIS MEANS

"Hey Laravel, when anyone asks for a `QuestionTypeRegistry`, run this closure once, build a registry pre-populated with all 5 types, and cache it for the rest of the request." That's what `singleton` does.

The 5 types — what each does

Type	config holds	Options table	Scoring
BinaryType	<code>{correct: 'yes' 'no'}</code>	None (deletes existing)	Match correct answer → full marks
SingleChoiceType	<code>{}</code> (empty)	One row per option, one marked correct	Pick correct option → full marks
MultipleChoiceType	<code>{scoring: 'strict' 'partial'}</code>	One row per option, multiple may be correct	Strict (all-or-nothing) or partial credit
NumberType	<code>{expected, tolerance}</code>	None	<code>abs(given - expected) ≤ tolerance</code> → full marks
TextType	<code>{expected, match}</code>	None	Match modes: exact, ci, contains, regex

How adding a new type would work

1. Create `app/QuestionTypes/Types/DatePickerType.php` implementing the interface.
2. Add one line in `QuestionTypeServiceProvider` : `->register(new DatePickerType)` .
3. Create the 3 Blade partials it points to.

Zero changes to controllers. Validation auto-extends. Dropdowns auto-extend. That's the Open/Closed Principle.

11. The Service Container & Dependency Injection

What is the service container?

Laravel's **service container** is a giant PHP factory. You can ask it: "Give me an instance of class X." It will:

1. Look at the class's constructor.
2. For each parameter, recursively ask the container for that type.
3. Build the whole tree of dependencies and return the assembled object.

Why this matters here

Look at the controller:

```
public function __construct(private QuestionTypeRegistry $registry) {}
```

You never write `new QuestionController(new QuestionTypeRegistry())`. Laravel does it for you, every request. **Decoupled and testable.**

What is dependency injection?

Instead of a class building its own dependencies (`$this->db = new Database()`), they're **passed in**. The class doesn't care where they came from. In tests, you can pass mocks instead of real things.

Singleton vs bind

Method	Behavior
<code>\$this->app->singleton(X, fn)</code>	Run <code>fn</code> once per request, cache result. Same instance every time.
<code>\$this->app->bind(X, fn)</code>	Run <code>fn</code> every time X is requested. Fresh instance each time.

The registry is a singleton — no need to rebuild it five times in one request.

12. File Uploads & Storage

The upload flow

```
$imagePath = $request->hasFile('image')
    ? $request->file('image')->store('questions', 'public')
    : null;
```

- `$request->hasFile('image')` — was a file uploaded under the `image` input name?
- `$request->file('image')` — get the `UploadedFile` object.
- `->store('questions', 'public')` — save it under the `questions/` folder on the **public** disk.
Returns the relative path with a random filename.

What is "the public disk"?

Laravel's filesystem is configured with named "disks" (in `config/filesystems.php`). The `public` disk maps to `storage/app/public`, which is symlinked to `public/storage` via `php artisan storage:link`. This makes uploaded files accessible at URLs like `/storage/questions/abc123.jpg`.

Why store the path, not the file?

Files belong on disk. The DB just stores a string path so we know where to find them. Tiny rows, fast queries.

Cleanup

When updating or deleting, the controller manually removes old files via `Storage::disk('public')->delete($path)`. The DB doesn't track files, so we have to.

13. Validation

```
$data = $request->validate([
    'title' => ['required', 'string', 'max:255'],
    'description' => ['nullable', 'string'],
]);
```

How it works:

1. You pass an array of rules.
2. If validation fails, Laravel **auto-redirects back** with errors and the user's input flashed to session. The view can show errors via `$errors`.
3. If validation passes, you get back an array of only the validated fields. Anything else the user sent is silently dropped.

Common rules used in this app

Rule	Meaning
required	Field must be present and not empty
nullable	Field may be null/missing
string	Must be a string
integer	Must be an integer
min:1, max:255	Numeric/length bounds
url	Must be a valid URL
image	Must be an image (jpg/png/gif/etc.)
in:a,b,c	Must be one of the listed values

14. Views & Blade

Blade is Laravel's templating engine. It looks like HTML with sprinkles of `{{ }}` and `@directives`.

The basics

Syntax	What it does
<code>{{ \$name }}</code>	Echo with HTML escaping (XSS-safe)
<code>{!! \$html !!}</code>	Echo raw HTML (use carefully)
<code>@foreach(\$items as \$item)</code>	Loop
<code>@if(...) @endif</code>	Conditional
<code>@include('partials.foo')</code>	Include another template
<code>@extends('layouts.app')</code>	Inherit a layout
<code>@section('content')</code>	Define a section in the layout

How types render type-specific UI

The view does this:

```
@include($registry->for($question->type)->inputView())
```

For a binary question that's `questions.input.binary`; for multiple choice it's `questions.input.multiple_choice`. **Same view code works for any type** — it just asks the type which partial to use.

15. The Complete Request Walkthrough

Let's trace one full user journey: a student submits a quiz attempt.

Setup

Quiz #5 has 3 questions: a binary, a multiple choice, and a number. Student starts the attempt, answers them, hits Submit.

Step 1: Browser sends the request

```
POST /attempts/12/submit
Content-Type: application/x-www-form-urlencoded
Body: answers[1]=yes&answers[2][]=4&answers[2][]=5&answers[3]=42
```

Step 2: Laravel boots and routes

Router finds: `Route::post('attempts/{attempt}/submit', [AttemptController::class, 'submit'])`.

Step 3: Controller is built

Container sees `AttemptController` needs a `QuestionTypeRegistry`. It gets the cached singleton (already populated with 5 types).

Step 4: Route-model binding

Laravel sees `{attempt}` = "12" and the method signature is `Attempt $attempt`. It runs `Attempt::findOrFail(12)` and passes the model.

Step 5: Guard against double-submit

```
abort_if($attempt->isSubmitted(), 403, ...) — if already submitted, return 403.
```

Step 6: Eager-load the data we need

```
$attempt->load('quiz.questions.options') — three nested levels in one go.
```

Step 7: Loop and score

For each question:

- **Question 1 (binary):** `$type = registry->for('binary')` . Normalize: `['value' => 'yes']` . Evaluate against `config['correct']` . Save Answer with marks.
- **Question 2 (MC):** Normalize: `['option_ids' => [4, 5]]` . Evaluate (strict or partial). Save.
- **Question 3 (number):** Normalize: `['value' => 42.0]` . Evaluate against expected $\pm$ tolerance. Save.

Step 8: Update the attempt with totals

```
$attempt->update([
  'submitted_at' => now(),
  'score' => $total,
  'max_score' => $attempt->quiz->totalMarks(),
]);
```

Step 9: Redirect

`return redirect()->route('attempts.show', $attempt);` — sends a 302 redirect to `/attempts/12` .

Step 10: Browser follows the redirect

New `GET /attempts/12` request → `show()` method → renders the result page with each question, the student's answer, and marks awarded. Done.

16. Glossary of Terms

Term	Definition
Framework	Pre-written code that solves common web problems (routing, DB, etc.)
MVC	Model-View-Controller architectural pattern
ORM	Object-Relational Mapping — treat DB rows as objects
Eloquent	Laravel's ORM
Migration	Versioned PHP file that creates/changes DB schema
Model	PHP class representing a DB table
Controller	Class with methods that handle HTTP requests
Route	Mapping of URL+verb → controller method
Route-model binding	Auto-fetch model from URL parameter
Middleware	Layer that processes requests/responses (auth, CSRF, etc.)
Service Container	Laravel's IoC container — builds objects with dependencies
Service Provider	Bootstrap class that registers things in the container
Dependency Injection	Pass dependencies in instead of building them inside
Facade	Static-like accessor for a container service (e.g., <code>Storage::</code>)
Blade	Laravel's templating engine for views
Eager loading	Load relationships up-front to avoid N+1
N+1 query problem	1 query for parents, N more queries for each child relationship
Mass assignment	Setting many model attributes from an array; controlled by <code>\$fillable</code>
Cast	Auto-conversion of DB values to PHP types (e.g. JSON → array)
Migration	Code-based DB schema versioning
Foreign key	DB constraint linking rows across tables
Cascade delete	Auto-delete child rows when parent is deleted
Strategy Pattern	Encapsulate interchangeable algorithms behind a common interface
Registry	A lookup map from key → object/class
Singleton	One instance, reused
Open/Closed Principle	Open for extension, closed for modification
POST/Redirect/GET	After form POST, redirect to a GET to prevent re-submit on refresh
CSRF token	Cross-Site Request Forgery protection token Laravel adds to forms

Flash session	Data stored in session for one request only (e.g. success messages)
Artisan	Laravel's CLI tool

17. Interview Questions with Answers

Project & Architecture

Q: Walk me through this project.

A: It's a dynamic quiz app in Laravel. Admins create quizzes and add questions of 5 different types — binary, single choice, multiple choice, number, text. Students take quizzes and get auto-graded. The interesting design is the **strategy pattern via a type registry**: each question type is its own class implementing a common interface; the controller never has an if/else over types. All questions live in one table with a JSON config column for type-specific settings.

Q: Why didn't you use polymorphic models or one table per question type?

A: Polymorphic models work but add complexity (morphTo, morphMany, two columns per relation). Separate tables per type would mean joining 5 tables and complicate the editor and scoring. **Single table with a JSON config column** is the simplest design that supports unbounded types. The trade-off is you can't easily query into JSON in vanilla SQL, but for read-with-the-row settings, it's fine.

Q: How would you add a new question type tomorrow?

A: Three steps: (1) Create a class implementing `QuestionType` with the 9 interface methods. (2) Add one line in `QuestionTypeServiceProvider` to register it. (3) Create the 3 Blade partials for editor / input / review. **No controller changes**. Validation auto-extends because the rule is built from the registry.

Laravel Basics

Q: What happens between the URL and the page?

A: Browser sends HTTP request → web server passes to `public/index.php` → Laravel boots, registers service providers → middleware runs → router matches the route → controller is built (DI) → controller method runs (talks to DB, validates) → returns a view or redirect → response goes back through middleware → browser renders.

Q: What is MVC?

A: Model-View-Controller. Model = data + business rules (Eloquent models). View = presentation (Blade templates). Controller = traffic cop that takes requests, asks the model, sends data to the view. Separation of concerns.

Q: What does `Route::resource` do?

A: It registers 7 RESTful routes for a resource: index, create, store, show, edit, update, destroy. Saves writing them by hand and enforces convention.

Q: What is route-model binding?

A: When a URL has a parameter like `{quiz}` and the controller method declares `Quiz $quiz`, Laravel automatically runs `Quiz::findOrFail(...)` with the URL value and passes the model. If not found, it 404s. You don't write the lookup yourself.

Q: What does `QuestionController::class` mean?

A: It's a PHP language construct that returns the fully-qualified class name as a string. Used instead of typing the string manually so the IDE can autocomplete and refactor it. It's NOT creating an instance.

Q: What does `->name('something')` do on a route?

A: Gives the route a label so you can reference it via `route('something', $params)` elsewhere instead of hardcoding the URL. If the URL ever changes, every reference still works.

Eloquent & ORM

Q: What is an ORM and why use one?

A: Object-Relational Mapping. It lets you treat DB rows as PHP objects instead of writing raw SQL. Cleaner code, less repetition, automatic SQL generation, type safety, easy relationships, protection against SQL injection by default.

Q: What's the N+1 problem and how do you solve it?

A: You fetch N parent rows in 1 query, then loop and access a relationship on each — triggering N more queries. For 100 quizzes, that's 101 SQL hits. Solve with eager loading:

`Quiz::with('questions')->get()` uses just 2 queries total.

Q: What does `withCount` do?

A: Adds a count of related rows as a new property without loading them.

`Quiz::withCount('questions')` attaches `questions_count` via a single SQL subquery. Efficient.

Q: What is `$fillable` ?

A: An allow-list of fields that can be mass-assigned via `create()` or `update()` . Without it, an attacker could POST extra fields like `is_admin=1` and Eloquent would happily save them. **Mass assignment protection.**

Q: What are casts on a model?

A: Auto-convert DB values to PHP types. `'config' => 'array'` means the JSON column is automatically decoded to a PHP array on read and encoded on write. `'is_correct' => 'boolean'` turns the DB's `0 / 1` into `true / false` .

Q: What's the difference between `hasMany` and `belongsTo` ?

A: `hasMany` is the parent side (Quiz has many Questions). `belongsTo` is the child side (Question belongs to Quiz). The foreign key (`quiz_id`) lives on the child table.

Q: What does `$quiz->questions()->create([...])` do that `Question::create([...])` wouldn't?

A: It auto-fills `quiz_id` from `$quiz` . The relationship knows the foreign key, so you don't have to set it manually.

Q: When would you use `load` vs `loadMissing` vs `fresh` ?

A: `load` always queries (use after creation). `loadMissing` only queries if not already loaded (avoids redundancy). `fresh` returns a new instance freshly fetched from DB (use after data may have changed under you).

The Type System

Q: Why a registry instead of a switch statement?

A: A switch couples the controller to every concrete type. Adding a type means editing the switch in every method that uses it. With a registry, the controller depends only on the abstraction. Adding a type is a one-class, one-line change. Open/Closed Principle.

Q: What design pattern is this?

A: It's the **Strategy Pattern** — interchangeable algorithms behind a common interface. The registry is a **Service Locator** for those strategies. The composition is a textbook way to eliminate type-based conditionals.

Q: Where does the registry get populated?

A: In `QuestionTypeServiceProvider::register()`. It calls `$this->app->singleton` to bind the registry as a singleton, with a closure that creates the registry and chains `register()` calls for all 5 types. Runs once during boot.

Q: Why is the registry a singleton?

A: No reason to rebuild it multiple times in one request — the type instances are stateless. A singleton means one instance, shared by everyone who asks. Saves work and memory.

Service Container & DI

Q: What is the service container?

A: Laravel's IoC (inversion of control) container. It builds objects and injects dependencies. You ask it for a class; it inspects the constructor, resolves each parameter recursively, and hands you a fully built instance.

Q: What is dependency injection?

A: Passing dependencies in via constructor (or method) instead of building them inside the class. Decouples the class from concrete implementations and makes testing easy — pass mocks in tests.

Q: How is the registry injected into the controller?

A: The constructor declares `private QuestionTypeRegistry $registry`. When Laravel needs to call the controller, the container builds it, sees the parameter type, looks up the singleton binding, and passes the registry instance.

HTTP & Forms

Q: What is the POST/Redirect/GET pattern?

A: After a form POST that mutates data, return a redirect to a GET URL instead of rendering directly. This way, refreshing the resulting page doesn't re-submit the form. Standard web hygiene.

Q: What does `->with('status', 'message')` do?

A: Flashes data to the session for the next request only. The view on the redirected-to page can read `session('status')` to show a one-time success message.

Q: What is CSRF and how does Laravel handle it?

A: Cross-Site Request Forgery. An attacker tricks a logged-in user into submitting a request. Laravel adds a hidden token to every form (via `@csrf`) and verifies it on POST/PUT/DELETE via middleware. No matching token → 419 error.

Q: How does file upload work in Laravel?

A: The form needs `enctype="multipart/form-data"`. The file arrives as an `UploadedFile`. `$request->file('image')->store('folder', 'public')` saves it to the public disk and returns the relative path, which you store in the DB.

Validation

Q: How does `$request->validate(...)` work?

A: If validation fails, Laravel auto-redirects back with errors and old input flashed to session. If it passes, you get an array of only the validated fields. Anything else is dropped.

Q: Why generate the `type` validation rule from the registry?

A: So adding a new type doesn't require updating the validation rule. The rule becomes `in:binary,single_choice,multiple_choice,number,text` dynamically. One source of truth.

Database

Q: What is a migration?

A: A versioned PHP file that defines a schema change. `php artisan migrate` applies them in order. Lets every developer and server have an identical schema, with rollback support.

Q: What does `foreignId('quiz_id')->constrained()->cascadeOnDelete()` do?

A: Creates an unsigned bigint `quiz_id`, adds a foreign key constraint pointing to `quizzes.id`, and tells the DB to auto-delete this row when the referenced quiz is deleted.

Q: Why a composite index on `(quiz_id, position)` ?

A: The most common query is "questions for this quiz, ordered by position". A composite index makes both the WHERE and ORDER BY use the index, giving $O(\log n)$ lookup instead of full scan + sort.

Q: Why store `config` as JSON instead of separate columns?

A: Different question types need different settings. A binary needs `{correct}`; a number needs `{expected, tolerance}`; a text needs `{expected, match}`. Adding columns for every possible setting bloats the schema. JSON gives schema flexibility per type while keeping one table.

Scoring & Logic

Q: How does multiple-choice scoring work?

A: Two modes via `config['scoring']`: **strict** means the picked set must equal the correct set exactly — full marks or zero. **partial** awards $(\text{right} - \text{wrong}) / \text{total}$ of the marks, floored at 0.

Q: How does number scoring work?

A: $\text{abs}(\text{given} - \text{expected}) \leq \text{tolerance}$ → full marks; else zero. The tolerance is per-question, set by the admin.

Q: What match modes does text support?

A: Four: `exact` (case-sensitive equality), `ci` (case-insensitive), `contains` (substring, case-insensitive), `regex` (regex match). Configurable per question.

Improvements & Trade-offs

Q: What would you improve if you had more time?

A: (1) Wrap `store` and `submit` in DB transactions so partial failures roll back. (2) Add authorization (Policies) — currently anyone with a URL can edit. (3) Add per-type form-request validation classes instead of doing common validation in the controller. (4) Add tests for each type's `evaluate` method. (5) Add a `QuestionType` abstract base class to share common boilerplate.

Q: What if scoring becomes more complex (e.g. partial number credit, fuzzy text)?

A: The architecture handles it: extend the type's `evaluate` method, add fields to `buildConfig`, update the editor partial. Nothing else changes.

Q: What if a question type needs custom validation rules?

A: Add a `validationRules(): array` method to the `QuestionType` interface. Each type returns its own rules. The controller merges them with the base rules. Single Responsibility maintained.

18. Final Cheat Sheet & Talking Points

The 60-second project pitch

"It's a dynamic quiz system in Laravel with five question types — binary, single choice, multiple choice, number with tolerance, and text with match modes. The key design decision was using the **strategy pattern**: each question type is its own class implementing a common interface, registered in a service-container singleton. The controller looks up the right handler by slug and delegates — there's no `if/else` over types anywhere. All questions live in one table with a JSON config column for type-specific settings. Adding a new question type tomorrow is a one-class, one-line change with zero modifications to existing code. That's the Open/Closed Principle made concrete."

Five soundbites to memorize

1. "The registry is a singleton dictionary mapping slugs to type handlers."
2. "The controller programs against the `QuestionType` interface, never against concrete classes."
3. "Validation rules and dropdown options are generated from the registry, so new types auto-extend the system."
4. "Single-table polymorphism with a JSON config column gives flexibility without a schema explosion."
5. "Eager loading via `load('questions.options')` avoids the N+1 problem in views."

If you only remember three things

#	Concept	Why it matters
1	Strategy + Registry pattern	Eliminates type-based <code>if/else</code> , enables open/closed extension
2	Eloquent relationships + eager loading	Clean code, avoids N+1, auto-handles foreign keys
3	Dependency injection via service container	Decoupling, testability, no manual <code>new</code>

If they ask "What did you learn?"

"Building this taught me to push complexity into focused classes and let the framework's container wire them up. Once the strategy interface was solid, the controller stayed boring — and that's exactly what I want from a controller. The 'magic' is in the design, not in clever code."

The body language tip

When they show you a piece of code and ask "explain this", **don't immediately speak**. Read it for 5 seconds. Then start with the *why*: "This is the controller method that handles X. The interesting bit here is..." That framing instantly sounds senior.

YOU'VE GOT THIS

Go in calm. The architecture genuinely is good — strategy pattern, single-table polymorphism, eager loading, dependency injection. Speak to the *why*, not just the what, and you'll do well. Good luck!